

Universidade Federal de Viçosa
Campus Rio Paranaíba

Lucas Souza e Silva — 7561

ANÁLISE COMPARATIVA DE ALGORITMOS DE ORDENAÇÃO

Rio Paranaíba - MG

2026

Universidade Federal de Viçosa
Campus Rio Paranaíba

Lucas Souza e Silva — 7561

ANÁLISE COMPARATIVA DE ALGORITMOS DE ORDENAÇÃO

Trabalho apresentado para obtenção de créditos na disciplina SIN213 — Projeto de Algoritmos da Universidade Federal de Viçosa - Campus de Rio Paranaíba, ministrada pelo Professor Pedro Moisés de Souza.

Rio Paranaíba - MG
2026

1 RESUMO

Ordenar é uma das operações mais frequentes em computação, e os algoritmos que a realizam diferem muito em custo conforme o tamanho e a ordem inicial dos dados. Conhecer esse comportamento é o que permite escolher o algoritmo certo para cada situação. Este trabalho implementa o Insertion Sort em linguagem C e mede o seu tempo de execução em dezoito combinações de entrada: seis tamanhos, de 10 a 1.000.000 de elementos, em três tipos de instância — crescente, decrescente e aleatória. Os resultados confirmaram a análise teórica. Com a entrada já ordenada, o algoritmo é linear e não registrou tempo mensurável em nenhum tamanho. Com entradas decrescente e aleatória, ele é quadrático: multiplicar o número de elementos por dez multiplicou o tempo por cerca de cem, e a entrada decrescente, que é o pior caso, custou o dobro da aleatória. O Insertion Sort mostrou-se adequado a conjuntos pequenos ou quase ordenados e impraticável para entradas grandes em ordem qualquer, em que um milhão de elementos levou até dois minutos para ser ordenado.

Sumário

1	RESUMO	1
2	INTRODUÇÃO	3
3	CAPÍTULO 1 — INSERTION SORT	4
3.1	Introdução ao algoritmo	4
3.1.1	Como o algoritmo funciona	4
3.1.2	Exemplo passo a passo	4
3.1.3	Pseudocódigo e custo por linha	5
3.1.4	Implementação em linguagem C	6
3.2	Teste de mesa manual	7
3.3	Análise e complexidade do algoritmo	8
3.3.1	Melhor caso	8
3.3.2	Caso médio	8
3.3.3	Pior caso	9
3.4	Tabelas e gráficos dos resultados	11
3.4.1	Ambiente de teste	11
3.4.2	Tempos medidos	11
3.4.3	Gráfico dos tempos	11
3.4.4	Discussão dos resultados	12
3.5	Conclusão do capítulo	14
4	REFERÊNCIAS BIBLIOGRÁFICAS	15
5	Função para calcular o tempo	16

2 INTRODUÇÃO

Em várias situações do dia a dia nos deparamos com a necessidade de trabalhar com dados devidamente ordenados. Ao procurar um contato numa lista telefônica, por exemplo, tudo depende de os nomes estarem em ordem alfabética; sem isso, a busca seria elemento por elemento. Não é difícil perceber que a ordenação está presente em boa parte do que se faz em computação, seja como fim, seja como etapa de outra tarefa, e por isso ela é tratada como um dos problemas fundamentais da área (CORMEN *et al.*, 2012).

Existem muitos algoritmos de ordenação, e eles diferem bastante entre si: alguns são simples de entender e implementar, mas ficam lentos à medida que a entrada cresce; outros são mais elaborados e escalam melhor. O custo de cada um também depende da ordem em que os dados chegam — o mesmo algoritmo pode ser rápido para uma entrada quase ordenada e lento para uma entrada invertida. Conhecer esse comportamento é o que permite escolher o algoritmo adequado para cada situação.

O objetivo deste trabalho é implementar algoritmos de ordenação em linguagem C e analisar o seu desempenho de duas maneiras que se complementam: pela análise de complexidade, que prevê como o custo cresce com o tamanho da entrada, e pela medição experimental, que registra o tempo real de execução em instâncias de diferentes tamanhos e tipos. Cada algoritmo ocupa um capítulo próprio, que o apresenta, analisa a sua complexidade, discute os experimentos realizados com ele e conclui sobre o seu desempenho.

O Capítulo 1 trata do **Insertion Sort**, ou ordenação por inserção. É um dos métodos mais simples que existem: percorre o vetor da esquerda para a direita e, a cada elemento, procura o lugar dele no trecho já ordenado, deslocando os maiores para abrir espaço. Sua simplicidade tem um preço, o custo quadrático nos casos médio e pior — mas ele tem também um melhor caso linear, quando a entrada já está ordenada, e é justamente essa diferença entre os casos que os experimentos do capítulo tornam visível.

3 CAPÍTULO 1 — INSERTION SORT

3.1 Introdução ao algoritmo

3.1.1 Como o algoritmo funciona

O Insertion Sort, ou ordenação por inserção, é um dos métodos de ordenação mais simples que existem. Ele funciona como quem organiza cartas na mão: as cartas da esquerda já estão em ordem e, a cada carta nova, procura-se o lugar dela nesse trecho, afastando as demais para abrir espaço. É com essa mesma analogia que Cormen *et al.* (2012) apresentam o método.

O algoritmo percorre o vetor da segunda posição até a última. A cada passagem vale uma regra: tudo o que está à esquerda da posição atual já se encontra ordenado. Cormen *et al.* (2012) chamam essa regra de invariante de laço, e é dela que sai a prova de que o algoritmo funciona.

Uma passagem faz o seguinte. O elemento da posição atual é copiado para uma variável auxiliar, o que deixa a célula livre. Esse valor é então comparado com os elementos à esquerda, um a um, da direita para a esquerda. Todo elemento maior que ele é empurrado uma casa para a direita. A comparação para assim que aparece um elemento menor ou igual, ou quando o início do vetor é alcançado. O valor guardado no auxiliar entra na casa que sobrou. Ao fim da passagem, o trecho ordenado ganhou um elemento; ao fim de todas, ele é o vetor inteiro.

Duas características do método importam para a análise que vem adiante. A primeira é que ele ordena *in-place*: não usa um segundo vetor. Fora o auxiliar e os índices, nada cresce junto com o tamanho da entrada, e por isso o espaço extra é $O(1)$. A segunda é que ele é estável: um elemento só é empurrado quando é estritamente maior que o valor a ser inserido, de modo que dois elementos iguais nunca trocam de ordem entre si (ZIVIANI, 2011).

3.1.2 Exemplo passo a passo

A Tabela 1 mostra esse processo em um vetor de cinco elementos, exibindo o estado dele ao fim de cada passagem. Nela, a passagem $j = 3$ não move nada, enquanto a passagem

$j = 4$ move três elementos. O trabalho de cada passagem não é fixo: depende de quão longe o elemento está do seu lugar definitivo. É dessa variação que nascem o melhor e o pior caso.

Tabela 1: Insertion Sort aplicado ao vetor $[7, 3, 9, 2, 5]$. Em negrito, o trecho já ordenado ao fim de cada passagem.

Passagem	Vetor	O que ocorreu
inicial	7 3 9 2 5	apenas o 7 está ordenado
$j = 2$	3 7 9 2 5	o 7 desloca; o 3 vai ao início
$j = 3$	3 7 9 2 5	o 9 já é maior que o 7
$j = 4$	2 3 7 9 5	o 2 percorre três elementos
$j = 5$	2 3 5 7 9	o 5 para logo após o 3

3.1.3 Pseudocódigo e custo por linha

A Tabela 2 apresenta o algoritmo em pseudocódigo e associa a cada linha o seu custo. O termo c_k é o custo de executar uma vez a linha k , e t_j , o número de vezes que o teste do laço interno é avaliado na passagem j . Os dois reaparecem na Seção 3.3.

Tabela 2: Pseudocódigo do Insertion Sort, com o custo e o número de execuções de cada linha.

<i>INSERTION-SORT(A)</i>		
PSEUDOCÓDIGO	CUSTO	VEZES
1- for $j \leftarrow 2$ to $\text{length}(A)$ do	c_1	n
2- $\text{key} \leftarrow A[j]$	c_2	$n - 1$
3- $i \leftarrow j - 1$	c_3	$n - 1$
4- while $i > 0$ and $A[i] > \text{key}$ do	c_4	$\sum_{j=2}^n t_j$
5- $A[i + 1] \leftarrow A[i]$	c_5	$\sum_{j=2}^n (t_j - 1)$
6- $i \leftarrow i - 1$	c_6	$\sum_{j=2}^n (t_j - 1)$
7- $A[i + 1] \leftarrow \text{key}$	c_7	$n - 1$

3.1.4 Implementação em linguagem C

A implementação usada neste trabalho é a versão tradicional apresentada em sala de aula, escrita em linguagem C e reproduzida no Código 1.

```
2      int i, j, aux;
4      for (i = 1; i < tamanho; i++) {
6          aux = vetor[i];
7          j = i - 1;
9          while (j >= 0 && vetor[j] > aux) {
10             vetor[j + 1] = vetor[j];
11             j--;
12         }
14         vetor[j + 1] = aux;
15     }
```

Código 1: Implementação do Insertion Sort em linguagem C.

3.2 Teste de mesa manual

TESTE DE MESA = (7 4 6 8 3 5)

j	CHAVE ← A[j]	i ← j-1	A[i+1] ← A[i]	i ← i-1	A[i+1] ← CHAVE
2	CHAVE ← A[2] CHAVE ← 4	i ← 1	A[2] ← 7	i ← 0	A[1] ← 4 4 7 6 8 3 5
3	CHAVE ← A[3] CHAVE ← 6	i ← 2	A[3] ← 7	i ← 1	A[2] ← 6 4 6 7 8 3 5
4	CHAVE ← A[4] CHAVE ← 8	i ← 3	NÃO ENTRA	-	A[4] ← 8 4 6 7 8 3 5
5	CHAVE ← A[5] CHAVE ← 3	i ← 4 i ← 3 i ← 2 i ← 1	A[5] ← 8 A[4] ← 7 A[3] ← 6 A[2] ← 4	i ← 3 i ← 2 i ← 1 i ← 0	A[1] ← 3 3 4 6 7 8 5
6	CHAVE ← A[6] CHAVE ← 5	i ← 5 i ← 4 i ← 3 i ← 2	A[6] ← 6 A[5] ← 5 A[4] ← 4 SAI DO LAÇO	i ← 4 i ← 3 i ← 2	A[3] ← 5 3 4 5 6 7 8

Figura 1: Teste de mesa manual do Insertion Sort.

3.3 Análise e complexidade do algoritmo

3.3.1 Melhor caso

O melhor caso ocorre quando o vetor já está em ordem crescente. A cada passagem, a primeira comparação do laço interno falha e nenhum elemento é deslocado: o algoritmo faz $n - 1$ comparações e nada mais. O custo é linear, $O(n)$. A dedução está na Figura 2.

$$\begin{aligned} \text{F.G: } T(m) &= c_1 \cdot m + c_2(m-1) + c_3(m-1) + c_4 \sum_{j=2}^m t_j + c_5 \sum_{j=2}^m (t_j - 1) + \\ & c_6 \sum_{j=2}^m 1 + c_7(m-1) \\ t_j = 1 &\rightarrow \sum_{j=2}^m t_j = m-1 \quad \sum_{j=2}^m (t_j - 1) = 0 \\ \text{Então,} \\ T(m) &= c_1 m + c_2(m-1) + c_3(m-1) + c_4(m-1) + c_7(m-1) \\ & \quad \quad \quad A \quad \quad \quad B \\ T(m) &= (c_1 + c_2 + c_3 + c_4 + c_7)m - (c_2 + c_3 + c_4 + c_7) \\ T(m) &= Am - B // \end{aligned}$$

Figura 2: Análise do melhor caso do Insertion Sort.

3.3.2 Caso médio

No caso médio, a entrada é aleatória e cada elemento precisa atravessar, em média, metade do trecho já ordenado. O laço interno executa cerca de $n^2/4$ vezes, e o custo é quadrático, $O(n^2)$. A dedução está na Figura 3.

$$\begin{aligned}
 x_j &= \frac{1}{2} \\
 \sum_{j=2}^m x_j &= \sum_{j=2}^m \frac{1}{2} \Rightarrow \frac{1}{2} \sum_{j=2}^m j = \frac{1}{2} \left(\frac{m(m+1)}{2} - 1 \right) \Rightarrow \frac{m^2 + m - 2}{4} \\
 \sum_{j=2}^m (x_{j-1}) &\Rightarrow \sum_{j=2}^m \left(\frac{1}{2} - 1 \right) = \sum_{j=2}^m \frac{1}{2} - \sum_{j=2}^m 1 \Rightarrow \frac{m^2 + m - 2}{4} - (m-1) \\
 &\Rightarrow \frac{m^2 + m - 2}{4} - \frac{4m - 4}{4} = \frac{m^2 - 3m + 2}{4} \\
 H(m) &= c_1 m + (c_2 + c_3 + c_7)(m-1) + c_4 \left(\frac{m^2 + m - 2}{4} \right) + (c_5 + c_6) \left(\frac{m^2 - 3m + 2}{4} \right) \\
 f(m) &= c_1 m + (c_2 + c_3 + c_7) m - (c_2 + c_3 + c_7) + \frac{c_4 m^2}{4} + \frac{c_4 m}{4} - \frac{c_4}{2} + \frac{(c_5 + c_6) m^2}{4} - \frac{3(c_5 + c_6) m}{4} + \frac{c_5 + c_6}{2} \\
 f(m) &= \frac{c_4 + c_5 + c_6}{4} m^2 + (c_1 + c_2 + c_3 + c_7 + \frac{c_4 - 3(c_5 + c_6)}{4}) m - c_2 - c_3 - c_7 - \frac{c_4}{2} + \frac{c_5 + c_6}{2}
 \end{aligned}$$

Figura 3: Análise do caso médio do Insertion Sort.

3.3.3 Pior caso

O pior caso ocorre quando o vetor está em ordem decrescente. Cada elemento é menor que todos os anteriores e atravessa o trecho ordenado inteiro: o laço interno executa cerca de $n^2/2$ vezes, o dobro do caso médio. O custo é quadrático, $O(n^2)$. A dedução está na Figura 4.

$$\begin{aligned}
 & t_j = j \\
 & \sum_{j=2}^m j = \left(\sum_{j=1}^m j \right) - 1 = \frac{m(m+1)}{2} - 1 \\
 & \sum_{j=2}^m (j-1) = \sum_{k=1}^{m-1} k = \frac{m(m-1)}{2} \\
 & T(m) = C_4 m + (C_1 + C_2 + C_7)(m-1) + C_4 \left(\frac{m(m+1)}{2} - 1 \right) + (C_5 + C_6) \cdot \left(\frac{m(m-1)}{2} \right) \\
 & T(m) = \overset{A}{\frac{C_4 + C_5 + C_6}{2}} m^2 + \overset{B}{\left(C_1 + C_2 + C_3 + C_7 + \frac{C_4 - C_5 - C_6}{2} \right)} m - \overset{C}{(C_2 + C_3 + C_4 + C_7)} \\
 & T(m) = Am^2 + Bm + C //
 \end{aligned}$$

Figura 4: Análise do pior caso do Insertion Sort.

3.4 Tabelas e gráficos dos resultados

3.4.1 Ambiente de teste

Todos os tempos foram medidos na mesma máquina: um notebook com processador Intel Core i7-13620H de 13^a geração e 15,7 GB de memória, sob Windows 11 Home. O programa foi compilado com o gcc 13.2.0 (MinGW-W64), usando as opções `-std=c99 -O2 -Wall`. A medição emprega a função `clock()`, da biblioteca `time.h`, e o cronômetro envolve apenas a chamada do `insertionSort`: a geração da instância e a gravação dos arquivos de entrada, de saída e de tempo ficam de fora da conta. Cada uma das dezoito combinações foi executada uma única vez.

3.4.2 Tempos medidos

A Tabela 3 reúne os tempos obtidos para os seis tamanhos de entrada e os três tipos de instância exigidos pelo trabalho.

Tabela 3: Tempo de ordenação do Insertion Sort, em segundos, nas dezoito combinações de tamanho e tipo de entrada.

n	Crescente	Decrescente	Random
10	0,000000	0,000000	0,000000
100	0,000000	0,000000	0,000000
1.000	0,000000	0,000000	0,000000
10.000	0,000000	0,012000	0,008000
100.000	0,000000	1,276000	0,613000
1.000.000	0,000000	119,339000	59,224000

3.4.3 Gráfico dos tempos

A Figura 5 mostra os mesmos tempos em forma de curva. As três séries partem do zero e assim permanecem até $n = 1.000$; a partir daí, a decrescente e a aleatória sobem juntas e guardam entre si uma distância que não muda até o último ponto, com a decrescente sempre

acima da aleatória e sempre pela mesma proporção. A crescente não sai do zero em nenhum dos seis tamanhos, e por isso acompanha o eixo horizontal de ponta a ponta.

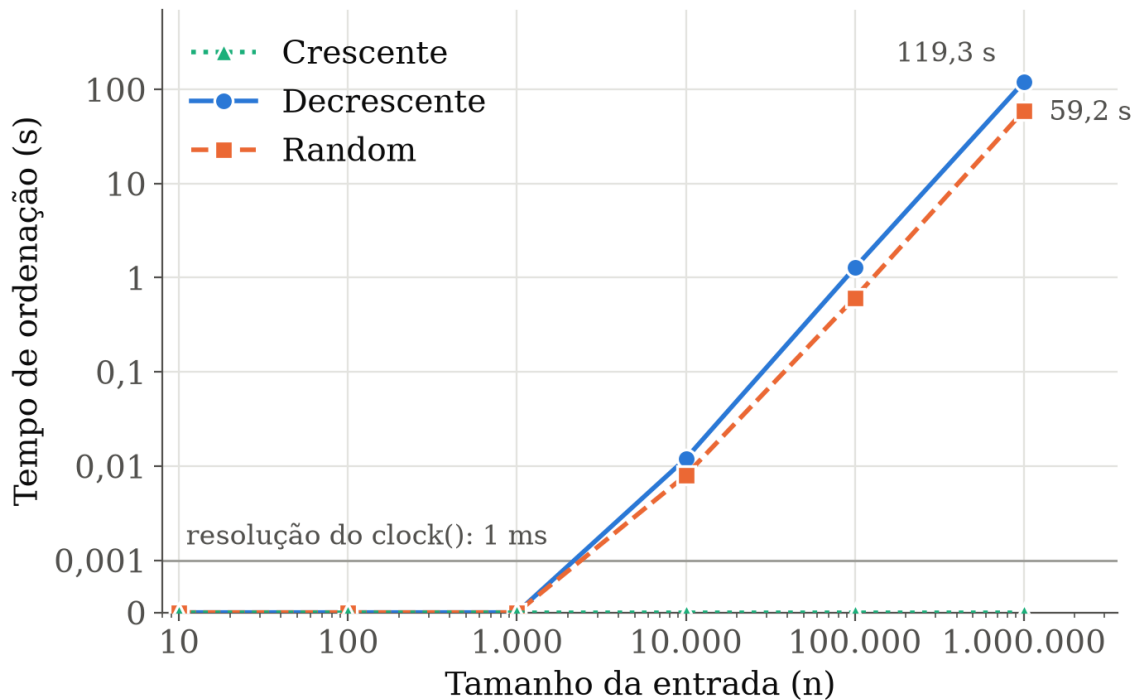


Figura 5: Tempo de ordenação em função do tamanho da entrada. O eixo do tempo é logarítmico acima de um milissegundo e linear abaixo disso, o que permite representar no gráfico as dez medições nulas.

3.4.4 Discussão dos resultados

Os resultados confirmam o que a análise da Seção 3.3 previa. A entrada crescente, que é o melhor caso, não registra tempo em nenhum tamanho: com o vetor já ordenado o laço interno nunca executa, e o custo linear fica abaixo do que o relógio consegue medir. A decrescente e a aleatória são ambas quadráticas — multiplicar n por dez multiplicou o tempo por cerca de cem nas duas — e a diferença entre elas está apenas na constante: a decrescente, que é o pior caso, custa o dobro da aleatória em todos os tamanhos medidos, porque cada elemento atravessa o trecho ordenado inteiro em vez de metade dele.

Os dez valores iguais a zero na tabela não são falha de medição. Nesta máquina, a função `clock()` só distingue variações a partir de um milissegundo, e tudo o que roda

mais rápido que isso é registrado como zero. É o caso da coluna Crescente inteira e dos três menores tamanhos das outras duas colunas.

O quadro geral é o de um algoritmo cujo desempenho depende mais da ordem da entrada do que do seu tamanho. Para vetores pequenos, ou já quase ordenados, o Insertion Sort é rápido o bastante para nem ser medido. Para vetores grandes em ordem qualquer, ele deixa de ser viável: com 1.000.000 de elementos, o mesmo código ou não registra tempo algum ou gasta quase dois minutos, e o que separa os dois extremos é apenas a disposição inicial dos números.

3.5 Conclusão do capítulo

Este capítulo apresentou o Insertion Sort, deduziu o seu custo nos três casos e mediu o seu tempo de execução em dezoito combinações de tamanho e tipo de entrada. A análise e os experimentos apontaram na mesma direção: o algoritmo é linear quando a entrada já está ordenada e quadrático nos demais casos, com o pior caso custando o dobro do caso médio.

Na prática, isso delimita bem onde o Insertion Sort se presta. Ele é simples, ordena no próprio vetor sem memória adicional e é estável, o que o torna uma boa escolha para conjuntos pequenos ou já quase ordenados — situações em que foi rápido a ponto de não ser medido. Para entradas grandes em ordem qualquer, porém, o crescimento quadrático o torna impraticável: ordenar um milhão de elementos aleatórios levou cerca de um minuto e, em ordem inversa, dois. Para essa faixa são necessários algoritmos de melhor ordem de crescimento.

4 REFERÊNCIAS BIBLIOGRÁFICAS

CORMEN, Thomas H. *et al.* **Algoritmos**: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

ZIVIANI, Nivio. **Projeto de algoritmos**: com implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2011.

5 Função para calcular o tempo

```
void operacoes(int tipo, int tamanho)
{
    clock_t start_t, end_t; //Variavel para guardar o tempo
    double tempoGasto;
    int * vetor = gerarSequencia(tipo, tamanho); //Gera sequencia de numeros
    //Salva a entrada de numeros
    salvarEntrada(tipo, tamanho, vetor);
    start_t = clock(); //Calcula o tempo atual
    insertionSort(vetor, tamanho); //Ordena o Vetor
    end_t = clock(); //Calcula o tempo apos ordenação
    tempoGasto = ((end_t - start_t) / (double)CLOCKS_PER_SEC); //Calcula diferença de
tempo
    salvarTempo(tipo, tamanho, tempoGasto); //Salva o tempo gasto
    //Salva a saída do programa
    salvarSaida(tipo, tamanho, vetor);
    //Libera a memoria
    free(vetor);
}
```